counting tackles the problem by storing a count of the number of threads accessing each node.

This may seem nice and straightforward, but it's quite hard to manage in practice. At first, you might think that something like `std::shared_ptr<>` would be up to the task; after all, it's a reference-counted pointer. Unfortunately, although some operations on `std::shared_ptr<>` are atomic, they aren't guaranteed to be lock-free. Although by itself this is no different than any of the operations on the atomic types, `std::shared_ptr<>` is intended for use in many contexts, and making the atomic operations lock-free would likely impose an overhead on all uses of the class. If your platform supplies an implementation for which `std::atomic_is_lock_free(&some_shared_ptr)` returns `true`, the whole memory reclamation issue goes away. Just use `std::shared_ptr<node>` for the list, as in the following listing.

**Listing 7.9. A lock-free stack using a lock-free `std::shared_ptr<>` implementation**

```
template<typename T>
class lock_free_stack
{
private:
    struct node
    {
        std::shared_ptr<T> data;
        std::shared_ptr<node> next;

        node(T const& data_):
            data(std::make_shared<T>(data_))
        {}
    };

    std::shared_ptr<node> head;
public:
    void push(T const& data)
    {
        std::shared_ptr<node> const new_node=std::make_shared<node>
(data);
        new_node->next=head.load();
        while(!std::atomic_compare_exchange_weak(&head,
                    &new_node->next,new_node));
    }
    std::shared_ptr<T> pop()
```